FOUNDATION_AUTH - Authentication and Authorization

Developer implementation guide: `FOUNDATION_AUTH_DEV_IMPL_GUIDE.md`.

1. Purpose

This foundation defines how every SOPHIX module authenticates users, authorizes API calls, and authenticates service-to-service HTTP calls.

Use this document when implementing:

- A user-facing REST API.
- An internal module-to-module REST API.
- Role checks for admin or backoffice actions.
- JWT validation middleware.
- Service-token validation middleware.

2. Ownership

FOUNDATION_AUTH owns:

- The central Identity Provider design.
- JWT issuer and validation rules.
- Shared role naming.
- Module middleware requirements.
- Service-to-service token pattern.
- Endpoint auth conventions.

FOUNDATION_AUTH does not own:

- External gateway callback signing, for example M-Pesa HMAC or bank callback verification.
- Module-specific endpoint role matrices. Each DD must declare its own required roles.
- Database encryption-at-rest. See `FOUNDATION_DB.md`.
- TLS design beyond assuming all internal HTTP traffic uses HTTPS.

3. Technology

Use Keycloak as the Identity Provider.

Logical endpoint:

```
https://idp.sophix.internal
```

Realm:

```
sophix
```

Token type:

```
OpenID Connect access token, signed with RS256
```

Modules verify JWTs locally. They must not call Keycloak on every request.

4. Runtime Architecture

```
sequenceDiagram
    participant User as User Browser
    participant UI as BSS UI
    participant KC as Keycloak
    participant API as SOPHIX Module API

    User->>UI: Open application
    UI->>KC: Redirect to login
```

```
KC->>KC: Authenticate by LDAP or local account
KC-->>UI: Access token + refresh token
UI->>API: Authorization: Bearer <jwt>
API->>API: Verify JWT signature and claims locally
API->>API: Check endpoint roles
API-->>UI: API response
```

5. Deployment Topology

Component	Requirement
Keycloak nodes	2 nodes behind NGINX
Keycloak DB	PostgreSQL database `keycloak` in DB cluster A
Persistent state	Shared PostgreSQL: realms, clients, users, roles, LDAP mappings
Runtime cache/session state	Keycloak embedded Infinispan cluster across both nodes
Node discovery	`jdbc-ping` for VM/bare-metal deployments; DNS/Kubernetes discovery if deployed on Kubernetes
User source	Corporate LDAP plus local Keycloak accounts
Public IdP URL	`https://idp.sophix.internal`
Realm	`sophix`
Token algorithm	RS256
Access token lifetime	1 hour
Refresh token lifetime	8 hours
SSO idle timeout	30 minutes
SSO max lifetime	10 hours

Two-node behavior:

- Either Keycloak node can serve login, token refresh, and OIDC metadata requests.
- Persistent IdP configuration is not replicated node-to-node; both nodes read and write the shared `keycloak` PostgreSQL database.
- Runtime session/authentication caches are clustered through Infinispan.
- If one Keycloak node fails, the load balancer routes traffic to the surviving node.
- If the `keycloak` database fails, new logins and refreshes fail. Already-issued JWTs continue to work in SOPHIX modules until they expire because modules validate JWTs locally.

NGINX Reverse Proxy

```
upstream keycloak_backend {
    server kc-1.sophix.internal:8080;
    server kc-2.sophix.internal:8080;
}

server {
    listen 443 ssl;
    server_name idp.sophix.internal;

    ssl_certificate     /etc/nginx/certs/idp.crt;
    ssl_certificate_key /etc/nginx/certs/idp.key;

    location / {
        proxy_pass         http://keycloak_backend;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto https;
        proxy_set_header   X-Forwarded-Host  $host;
        proxy_set_header   X-Forwarded-Port  443;

        proxy_next_upstream error timeout http_502 http_503 http_504;
        proxy_read_timeout 30s;
    }
}
```

Implementation rule: keep `X-Forwarded-Proto` and `X-Forwarded-Host`. Keycloak uses them when building browser redirect URLs.

Load-balancer recommendation: enable sticky sessions when available. Sticky sessions are not the durability mechanism; PostgreSQL and Infinispan provide that. Stickiness simply keeps a browser's login flow on the same node and reduces cross-node cache lookups.

Keycloak Environment

```
KC_DB=postgres
KC_DB_URL=jdbc:postgresql://pgbouncer-a.sophix.internal:6432/keycloak
KC_DB_USERNAME=keycloak
KC_DB_PASSWORD=...

KC_HOSTNAME=idp.sophix.internal
KC_HOSTNAME_STRICT=true
KC_HOSTNAME_STRICT_HTTPS=true

KC_PROXY=edge
KC_HTTP_ENABLED=true

KC_CACHE=ispn
KC_CACHE_STACK=jdbc-ping
```

Store the Keycloak realm export in the infra repository so the realm can be rebuilt consistently.

Minimum realm settings:

```
{
  "realm": "sophix",
  "enabled": true,
  "accessTokenLifespan": 3600,
  "ssoSessionIdleTimeout": 1800,
  "roles": {
    "realm": [
      { "name": "ADMIN" },
      { "name": "PRODUCT_MANAGER" },
      { "name": "BILLING_LEAD" },
      { "name": "READ_ONLY" }
    ]
  },
  "clients": [
    {
      "clientId": "bss-backoffice-ui",
      "enabled": true,
      "publicClient": true,
      "redirectUris": ["https://bo.sophix.internal/*"],
      "webOrigins": ["https://bo.sophix.internal"]
    }
  ]
}
```

6. JWT Contract

Every user-facing API call must include:

```
Authorization: Bearer <jwt>
```

Required token claims:

```
{
  "iss": "https://idp.sophix.internal/realms/sophix",
  "sub": "user_01HX2Y3Z4",
  "name": "Alice Oyuga",
  "email": "alice@wananchi.com",
  "preferred_username": "alice.oyuga",
  "roles": ["PRODUCT_MANAGER", "BILLING_LEAD"],
  "iat": 1715425380,
  "exp": 1715428980
}
```

Validation rules:

Rule	Description
AUTH-JWT-1	`Authorization` header must exist for protected user-facing endpoints.
AUTH-JWT-2	Header must use `Bearer <jwt>` format.
AUTH-JWT-3	JWT signature must validate against Keycloak public keys.
AUTH-JWT-4	`iss` must equal `https://idp.sophix.internal/realms/sophix`.
AUTH-JWT-5	`exp` must be in the future.

Rule	Description
AUTH-JWT-6	`roles` claim is the source of truth for user authorization.
AUTH-JWT-7	Missing required role returns HTTP 403.
AUTH-JWT-8	Invalid, missing, or expired token returns HTTP 401.

7. Module Implementation

Every module that exposes user-facing endpoints must install JWT middleware.

Startup behavior:

1. Fetch public keys from:

   ```
   https://idp.sophix.internal/realms/sophix/protocol/openid-connect/certs
   ```

2. Cache the public keys in memory.
3. Refresh public keys at least every 6 hours.
4. Refuse protected traffic until the first key fetch succeeds.

Request behavior:

1. Read `Authorization`.
2. Verify token signature.
3. Verify issuer and expiry.
4. Extract `sub`, `preferred_username`, `email`, and `roles`.
5. Attach an authenticated principal to request context.
6. Run endpoint role guard.

Spring Boot Example

```
@RestController
@RequestMapping("/api/services")
class ServiceController {
    @PreAuthorize("hasAnyRole('PRODUCT_MANAGER','ADMIN')")
    @PostMapping
    public ServiceDto create(@RequestBody CreateServiceRequest request) {
        return serviceCatalog.create(request);
    }
}
```

Spring security config should use Keycloak JWKs as the JWT decoder source.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://idp.sophix.internal/realms/sophix
```

Laravel Example

```
Route::post('/api/services', [ServiceController::class, 'store'])
    ->middleware(['auth:jwt', 'role:PRODUCT_MANAGER|ADMIN']);
```

Laravel middleware must:

- Validate RS256 signature.
- Validate `iss` and `exp`.
- Convert `roles` claim into the authorization context.

8. Role Catalog

Roles are created centrally in Keycloak and assigned directly or through LDAP group mapping.

Modules must not invent local role names. If a DD needs a new role, add it here and to the Keycloak realm export.

Role	Typical users	Main usage
ADMIN	System admins	Full platform administration
READ_ONLY	Auditors, analysts	Read-only access
PRODUCT_MANAGER	Product/catalog team	Service, package, bundle, pricing catalog admin
BILLING_LEAD	Billing leads	Billing configuration and oversight
APPROVER_L1	Branch staff	First-level approvals
APPROVER_L2	Branch managers	Second-level approvals
WELCOME_CALL_AGENT	Call center	Welcome-call user tasks
FIELD_TECH	Field technicians	Field work order actions
SUPERVISOR	Operations supervisors	Escalations and manual review
FRAUD_REVIEWER	Fraud team	Fraud and mismatch reviews
CYCLE_ADMIN	Billing operations	BIL-03 manual cycle actions
CYCLE_OPERATOR	Billing operations	BIL-03 read and monitoring
DUNNING_ADMIN	Billing operations	BIL-04 dunning overrides and program lifecycle
DUNNING_OPERATOR	Billing operations	BIL-04 read and review queues
WALLET_ADMIN	Billing operations	BIL-05 wallet suspension and manual balance actions
WALLET_OPERATOR	Billing operations	BIL-05 wallet read and reconciliation
BILLING_INTERNAL	Service accounts	Internal billing calls into workflow or wallet APIs
GATEWAY_INTERNAL	Gateway adapters	Gateway callback entry points, plus provider-specific verification

9. Service-to-Service Authentication

Internal module-to-module HTTP calls use a static service token.

Header:

```
X-Service-Token: <opaque-service-token>
```

Each receiving module stores a token allow-list in configuration.

Example:

```
serviceTokens:
  subscription-engine: ${SUBSCRIPTION_ENGINE_SERVICE_TOKEN}
  fulfillment: ${FULFILLMENT_SERVICE_TOKEN}
  billing: ${BILLING_SERVICE_TOKEN}
```

Validation rules:

Rule	Description
AUTH-SVC-1	Internal endpoints must require `X-Service-Token` unless explicitly public.
AUTH-SVC-2	Unknown token returns HTTP 401.
AUTH-SVC-3	Valid token maps to a caller identity such as `billing` or `subscription-engine`.
AUTH-SVC-4	Endpoint-level guards may restrict which caller identity is allowed.
AUTH-SVC-5	Tokens must be stored in environment variables or secret storage, not source code.
AUTH-SVC-6	Rotation requires updating caller and receiver secrets, then rolling restart.

Example internal request:

```
POST /api/subscriptions/sub_01HX/activate HTTP/1.1
Host: subscription.sophix.internal
X-Service-Token: ${FULFILLMENT_SERVICE_TOKEN}
Content-Type: application/json

{
  "operatorCode": "WIK",
  "activationTrigger": "SALES_FIELD",
  "correlationId": "act_ord_01HX"
}
```

10. Endpoint Auth Matrix Convention

Every DD that defines REST APIs must include an auth column.

Endpoint type	Required auth
Backoffice or customer user API	JWT bearer token plus role guard
Internal module API	`X-Service-Token`
External callback	Integration-specific verification, for example HMAC, OAuth, public key, IP allow-list
Health/readiness	Usually unauthenticated inside private network
Public metadata	Unauthenticated only if explicitly declared

An endpoint may support multiple auth paths. Example: an admin endpoint may accept either `ADMIN` JWT or a trusted internal service token. The middleware must accept any configured path and still attach caller identity to request context.

11. Error Handling

Condition	HTTP status	Error code
Missing JWT on protected user endpoint	401	`AUTH_TOKEN_MISSING`
Malformed bearer header	401	`AUTH_HEADER_INVALID`
Signature verification failed	401	`AUTH_TOKEN_INVALID`
Token expired	401	`AUTH_TOKEN_EXPIRED`
Missing required role	403	`AUTH_ROLE_FORBIDDEN`
Missing service token	401	`SERVICE_TOKEN_MISSING`
Unknown service token	401	`SERVICE_TOKEN_INVALID`
Valid service token but caller not allowed	403	`SERVICE_CALLER_FORBIDDEN`

Recommended response shape:

```
{
  "error": {
    "code": "AUTH_ROLE_FORBIDDEN",
    "message": "The authenticated principal is not allowed to call this endpoint.",
    "correlationId": "req_01HX..."
  }
}
```

Do not include token contents or secret values in logs or responses.

12. Resilience

Failure	Expected behavior
Keycloak unavailable	Existing valid JWTs continue working until expiry. New login and refresh fail.
Keycloak PostgreSQL unavailable	New login fails. Existing JWTs remain valid. Recover DB cluster A per `FOUNDATION_DB.md`.
One Keycloak node unavailable	Load balancer removes the failed node. Surviving node continues serving.
Infinispan cluster split/cache issue	Login/session behavior may degrade. Restart or rejoin the affected node after verifying DB health.
LDAP unavailable	LDAP-backed users cannot log in. Local service/admin accounts continue if configured locally.
JWK fetch fails during module startup	Module retries with backoff and does not accept protected traffic until keys are loaded.
JWK refresh fails after startup	Continue using previously cached keys until refresh succeeds. Alert if refresh remains failing.
Service token leaked	Rotate token, update caller and receiver secrets, restart or reload config.

13. Infrastructure Recommendations

Area	Recommendation
Load balancer	Use health checks and sticky sessions. Route only to ready Keycloak nodes.
Database	Run the `keycloak` PostgreSQL database on HA infrastructure. It is the critical dependency for new login and token refresh.
Admin console	Restrict to VPN/admin network. Do not expose broadly inside or outside the operator network.
Realm config	Keep realm/client/role exports in Git and apply through infra automation.
Clients	Use separate OIDC clients per UI/app, e.g. `bss-backoffice-ui`, `customer-portal`, `field-agent-pwa`.
Secrets	Store client secrets, service tokens, LDAP bind credentials, and DB passwords in the platform secret manager.
Certificates	Monitor TLS certificate expiry for IdP and reverse proxy endpoints.
Monitoring	Track node health, login error rate, refresh error rate, PostgreSQL latency, Infinispan cluster membership, LDAP failures, and JWK endpoint health.
Backups	Back up the `keycloak` database and realm exports. Test restore before production go-live.

14. Medium Budget Sizing

This is the recommended starting point for a budget-limited medium deployment. It is intentionally conservative: enough HA for authentication without overbuilding the IdP layer.

14.1 Server Dimensions

Server role	Count	CPU	RAM	Disk	Notes
Keycloak app node	2	2 vCPU each	4 GB each	50 GB SSD each	Runs Keycloak only. App nodes are stateless enough to replace quickly.
Keycloak PostgreSQL node	2	2-4 vCPU each	8 GB each	150 GB SSD each	Primary + replica. Prioritize this over app-node CPU.
NGINX/load balancer	1 minimum, 2 preferred	1-2 vCPU each	2 GB each	20 GB SSD each	Start with 1 if budget is tight; use 2 for LB HA.
Monitoring/log collector	Shared platform service	2 vCPU	4 GB	100 GB+	Can be shared with broader infra if already deployed.

Minimum practical baseline:

```
2 x Keycloak nodes:    2 vCPU / 4 GB RAM / 50 GB SSD
2 x PostgreSQL nodes:  2 vCPU / 8 GB RAM / 150 GB SSD
1 x NGINX/LB node:     1 vCPU / 2 GB RAM / 20 GB SSD
```

Preferred medium baseline:

```
2 x Keycloak nodes:    2 vCPU / 4 GB RAM / 60 GB SSD
2 x PostgreSQL nodes:  4 vCPU / 8 GB RAM / 200 GB SSD
2 x NGINX/LB nodes:    1 vCPU / 2 GB RAM / 20 GB SSD
```

Placement rules:

- Do not place both Keycloak nodes on the same physical host.
- Do not place the PostgreSQL primary and replica on the same physical host.
- If possible, do not place a Keycloak node on the same physical host as the active PostgreSQL primary.
- PostgreSQL disk must be SSD. Avoid HDD for the `keycloak` database.
- Keycloak app nodes can be rebuilt from config; PostgreSQL must be backed up and replicated.

14.2 Recommended OS

Use one of:

OS	Recommendation
Ubuntu Server 22.04 LTS or 24.04 LTS	Recommended default for small/medium teams because packages, docs, and ops skills are widely available.
Rocky Linux 9 / RHEL 9	Good choice if the operator standardizes on RHEL-family systems.

OS baseline:

- 64-bit server edition.
- Minimal install; no desktop packages.
- Automatic security updates enabled or patched through the operator's standard maintenance process.
- Time sync enabled with `chrony` or equivalent NTP.
- SSH access restricted to admin network/VPN.
- Host firewall enabled. Expose only required ports.

14.3 Disk Partitions

For Keycloak app nodes, keep partitioning simple:

Mount	Size	Purpose
`/`	20 GB	OS and packages
`/opt/keycloak`	15 GB	Keycloak installation, providers, deployment files
`/var/log`	10 GB	OS, NGINX/client logs if local
`/tmp`	5 GB	Temporary files

For a 50-60 GB Keycloak node disk:

```
/             20 GB
/opt/keycloak  15 GB
/var/log       10 GB
/tmp            5 GB
free            remaining
```

For Keycloak PostgreSQL nodes:

Mount	Size	Purpose
`/`	25 GB	OS and packages
`/var/lib/postgresql`	80-120 GB	PostgreSQL data directory
`/var/lib/postgresql/wal_archive`	30-50 GB	WAL archive staging if archived locally before backup copy
`/var/log`	10-20 GB	PostgreSQL and OS logs
`/backup`	Optional, 50 GB+	Local backup staging only; final backups should leave the server

For a 150 GB PostgreSQL node disk:

```
/                             25 GB
/var/lib/postgresql           90 GB
/var/lib/postgresql/wal_archive  25 GB
/var/log                      10 GB
free                          remaining
```

For a 200 GB PostgreSQL node disk:

```
/                             25 GB
/var/lib/postgresql          120 GB
/var/lib/postgresql/wal_archive  35 GB
/var/log                      15 GB
free                           remaining
```

Filesystem recommendation:

- Use `ext4` or `xfs`.
- Use LVM if the operator is comfortable managing it; otherwise use plain partitions to reduce operational complexity.
- Keep PostgreSQL data and WAL archive on SSD-backed storage.

- Do not store long-term backups only on the PostgreSQL node. Copy backups to separate backup storage.

14.4 JVM Settings

For 4 GB RAM Keycloak nodes:

```
JAVA_OPTS="-Xms1g -Xmx2g"
```

This leaves memory for the OS, file cache, networking, and Keycloak overhead.

14.5 Network Ports

Flow	Port	Exposure
User/UI to IdP through LB	443	Internal/user-facing network as required
LB to Keycloak node	8080 or 8443	Internal only
Keycloak to PostgreSQL/PgBouncer	6432	Internal only
Keycloak to LDAP	389 or 636	Internal only; prefer LDAPS 636
Admin SSH	22	Admin VPN/network only
Monitoring scrape	Operator standard	Monitoring network only

15. AWS Deployment Recommendations

This section applies only when deploying the IdP stack on AWS. It does not replace the on-premise baseline; it maps the same architecture to AWS managed/network primitives.

15.1 Recommended AWS Shape

Component	AWS service	Recommendation
Keycloak app nodes	EC2 Auto Scaling Group	Run 2 EC2 instances across 2 Availability Zones. Keep desired capacity at 2.
Load balancer	Application Load Balancer	Use HTTPS listener, target group health checks, and sticky sessions.
Keycloak PostgreSQL	Amazon RDS for PostgreSQL	Use Multi-AZ for production.
TLS certificates	AWS Certificate Manager	Use ACM certificate on the ALB.
Secrets	AWS Secrets Manager or SSM Parameter Store	Store DB password, LDAP bind credentials, service tokens, and Keycloak admin bootstrap credentials.
Logs	CloudWatch Logs	Ship Keycloak and system logs.
Metrics/alarms	CloudWatch	Alarm on unhealthy targets, login errors, RDS CPU/storage, and DB connection count.

15.2 EC2 Sizing

Budget medium start:

```
Keycloak EC2: 2 x t4g.medium or t3.medium
  vCPU/RAM:   2 vCPU / 4 GB RAM each
  Storage:    50-60 GB gp3 EBS each

If ARM compatibility is confirmed:
  Prefer Graviton instances such as t4g.medium or m7g.medium for better price/performance.

If ARM compatibility is not approved by the local ops team:
  Use t3.medium or m6i.large.
```

Preferred medium:

```
Keycloak EC2: 2 x m7g.large or m6i.large
  vCPU/RAM:   2 vCPU / 8 GB RAM each
  Storage:    60 GB gp3 EBS each
```

Use Ubuntu Server LTS or Amazon Linux 2023. Keep the AMI minimal and patched.

15.3 Load Balancer

Use an Application Load Balancer:

- Listener: HTTPS 443.
- Target type: instances or IPs.
- Health check path: Keycloak health/readiness endpoint.
- Stickiness: enabled on the target group.
- Idle timeout: tune to support login/redirect flow; default is usually acceptable, but verify with field-agent PWA and backoffice login.
- Security group: allow 443 from user/admin networks; allow target traffic only from ALB to Keycloak instances.

Sticky sessions are recommended to reduce cross-node cache lookups during login. They are not the persistence mechanism.

15.4 RDS Recommendation

Use Amazon RDS for PostgreSQL for the `keycloak` database:

```
Instance class:  db.t4g.medium minimum, db.m7g.large preferred
Storage:         100-200 GB gp3
Deployment:      Multi-AZ
Backups:         automated backups enabled, 7-30 day retention
Encryption:      enabled with KMS
Network:         private subnets only
```

Do not place the Keycloak database in a public subnet. Keycloak EC2 instances should connect to RDS through private subnets/security groups.

15.5 AWS Network Layout

Recommended VPC layout:

```
Public subnets:
  - ALB only

Private app subnets:
  - Keycloak EC2 nodes

Private DB subnets:
  - RDS PostgreSQL
```

Security group rules:

Source	Destination	Port	Purpose
User/admin networks	ALB	443	Browser login
ALB SG	Keycloak SG	8080 or 8443	Forward to Keycloak
Keycloak SG	RDS SG	5432	Database access
Keycloak SG	LDAP/VPN endpoint	636	LDAPS
Admin VPN/bastion	EC2 nodes	22	Admin access

15.6 AWS Notes

- Prefer private subnets for all app and DB resources.
- Use NAT Gateway or controlled egress only if instances need outbound package updates.
- Use IAM roles for EC2 access to CloudWatch/SSM; do not store AWS keys on servers.
- Use Systems Manager Session Manager instead of SSH where the operator supports it.
- Snapshot RDS before Keycloak major upgrades.
- Keep the Keycloak realm export in Git even when using RDS snapshots.

Reference docs:

- [Amazon RDS Multi-AZ deployments](https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html)
- [Application Load Balancer sticky sessions](https://docs.aws.amazon.com/elasticloadbalancing/latest/application/modify-target-group-health-settings.html)

- [Amazon EC2 instance types](https://docs.aws.amazon.com/ec2/latest/instancetypes/gp.html)

16. Operational Endpoints

Endpoint	Caller	Purpose
`GET /realms/sophix/protocol/openid-connect/certs`	Modules	Fetch public signing keys
`GET /realms/sophix/.well-known/openid-configuration`	Modules, UIs	Discover OIDC metadata
Keycloak admin console	IdP admins	Manage users, clients, roles, LDAP mapping

Full URLs use the IdP base:

```
https://idp.sophix.internal
```

17. Developer Checklist

Before exposing a new API:

- [ ] Decide whether the endpoint is user-facing, internal, external callback, or public.
- [ ] Add JWT role guard or service-token guard.
- [ ] Declare required auth in the DD API table.
- [ ] Use centralized role names from this document.
- [ ] Return 401 for unauthenticated requests.
- [ ] Return 403 for authenticated but unauthorized requests.
- [ ] Log caller identity, not raw tokens.
- [ ] Include `correlationId` in auth failure logs.
- [ ] Add tests for missing token, invalid token, expired token, missing role, and successful access.